开发者社区

DTK

DTK 26.10
DCC 使用手册

DTK-261000-019-0915-934

目　录

1 DCC 简介 1
1.1 DCC 介绍 1
1.2 DCC 支持概况 1
2 编译流程 1
3 DCC 编译选项 2
3.1 异构加速卡编译选项 2
3.2 DCC 默认处理流程 3
3.3 DCC 调试选项 4
3.4 DCC 环境变量支持 4
4 DCC 宏定义 5
4.1 功能检查宏 5
4.2 头文件检查宏 6
4.3 内置宏 6

开发者社区

版本更新

- DTK-24.04.2：
  - 创建文档。

开发者社区

专有名词解释

DTK	DCU ToolKit	对应 DCU 硬件软件开发套件，应用开发者可以通过 DTK 开发套件使程序运行到 DCU 硬件。
DCC	DCU Compiler Collection	DTK(DCU ToolKit)对应编译器工具链。
HIP	Heterogeneous-compute Interface for Portability	是异构并行编程模型编程框架，用于在不同的 DCU 平台上实现跨平台的高性能计算。
HIPCC	Heterogeneous-compute Interface for Portability Compiler Command	ROCm(Radeon Open Compute)软件平台中的编译工具,DCC 完全兼容 HIPCC。
CUDA	Compute Unified Device Architecture	NVIDIA 开发的一种并行计算平台和编程模型，它允许开发者利用 NVIDIA 的 GPU(图形处理器)来进行高性能的通用计算
OpenMP	Open Multi-Processing	是一种支持多平台的共享内存、多处理器（多线程）的规范与 API

开发者社区

1 DCC 简介

1.1 DCC 介绍

DCC 是针对 DCU 异构硬件平台编译器工具链，作为 DTK 开发工具链的一部分。DCC 支持将基于 C/C++的 HIP 源代码编译为不同目标机器语言，并在多种 DCU 平台上生成相应的可执行文件。它能够根据环境类型打印编译环境信息，向编译器传递适当的编译和库选项，并根据用户输入的编译选项进行调整。通过 DCC，开发者可以使用统一的编程模型在多个 DCU 平台上编写和运行高效代码，从而提高代码的可移植性和可维护性。

1.2 DCC 支持概况

DCC 为自主研发，充分结合 DCU 的核心优势，全面支持 DCU 所有平台，同时兼容 HIPCC 的所有功能，实现对 HIP 程序的高效编译和优化。DCC 支持 HIPCC 的主要功能，包括：

1. **DCU 内核编译**：DCC 能够高效地处理 HIP 程序中 DCU 代码，实现针对不同 DCU 架构自动优化，生成高性能二进制代码。
2. **统一内存管理**：DCC 完整继承 HIPCC 内存管理机制，支持统一内存访问模型，简化开发者处理设备内存和主机内存之间数据传输工作。
3. **API 兼容性**：DCC 在 API 层面与 HIPCC 完全兼容，支持所有 HIP API 调用。开发者无需对现有 HIP 代码进行修改，即可使用 DCC 进行编译和优化。

2 编译流程

使用 DCC 可以将 C 语言家族的代码编译为机器码，包括将 HIP、OpenMP 等异构代码编译为可在多种架构上运行的二进制指令。DCC 的编译流程主要包括以下步骤：

- **预处理（Preprocessing）**

编译器首先会对输入源文件使用 C/C++预处理器进行处理，展开宏、处理条件编译等预处理操作。

- **源码转换（Source Translation）**

预处理后的源文件将被转换为 LLVM bitcode 格式。在转换过程中，经过词

法分析、语法分析、语义分析等步骤后，将代码转化为 LLVM IR 中的调用指令，以支持在 CPU 和 DCU 之间的执行切换。LLVM IR 是一种低级汇编语言，适用于代码优化和目标代码生成。

- **设备端编译（Device Compilation）**

转换后的 LLVM bitcode 被传递给 LLVM 等设备端编译工具进行编译。LLVM 能够将 bitcode 编译成目标设备的原生机器码，同时执行优化和代码重组以提升性能。

- **优化（Optimization）**

DCC 利用 LLVM 的优化工具对生成的 IR 进行一系列优化，包括死代码删除、常量传播、控制流简化等。这些优化大幅提高代码性能并减小代码体积。

- **链接（Linking）**

设备端编译完成后，编译器将目标文件或库文件链接成最终的可执行程序。

3 DCC 编译选项

3.1 异构加速卡编译选项

- -fgpu-default-stream=<arg>

该选项用于指定 DCU 上下文默认流。参数<arg>可以为 legacy、per-thread，例如，以下命令将使用系统默认流：

```
dcc -fgpu-default-stream=legacy file.cpp
```

- -fgpu-flush-denormals-to-zero, -fno-gpu-flush-denormals-to-zero

这些选项控制 DCU 是否将非规格化浮点数归零。

- --gpu-instrument-lib=<arg>

指定 DCU 计算时使用的性能分析库。该选项会插入 DCU 相关性能计数器，用于度量程序在 DCU 上的执行情况。例如，以下命令将使用 libgpuinstrument.so 插入计数器：

```
dcc --gpu-instrument-lib=libgpuinstrument.so file.cpp
```

- --gpu-max-threads-per-block=<arg>

指定每个 DCU 块(block)中最大线程数。例如，以下命令将每个块最大线程数设置为 1024：

dcc --gpu-max-threads-per-block=1024 file.cpp

- -nogpuinc, -nocudainc

该选项控制编译时是否包含 NVIDIA CUDA 头文件。如果使用此选项，将不会包含 NVIDIA CUDA 头文件。

- -nogpulib, -nocudalib

控制是否链接 NVIDIA CUDA 库。如果使用此选项，将不会链接 NVIDIA CUDA 库。

- --offload-arch=<arg>, --no-offload-arch=<arg>

这些选项用于启用或禁用对特定设备的支持，并指定用于 OpenMP 卸载场景的目标设备架构。例如，以下命令将目标 DCU 架构指定为 gfx906：

dcc --offload-arch=gfx906 file.cpp

- --offload-device-only

用于仅编译在设备上运行的代码。此选项可避免在主机 CPU 上运行 offload 代码，从而提高计算性能。

- --offload-host-device

用于编译运行在主机和设备代码。

- --offload-host-only

用于编译仅在主机上运行的代码。

- --offload=<arg>

使用<arg>作为可卸载目标设备或主机名称。在使用多个卸载目标的情况下，<arg>应为卸载链中的特定目标名称。

- --amdgpu-arch-tool=<arg>

该选项指定用于生成目标机器代码指令集的 DCU 架构工具。该工具可根据指定的 DCU 硬件架构自动生成指令集，并将其传递给编译器以生成二进制代码。

3.2 DCC 默认处理流程

在默认情况下，DCC 不仅会保留用户输入的编译参数，还会自动添加一些编译选项 Cxxflags 和 Ldflags。Cxxflags 主要是-isystem 参数，用于指定系统头文件的搜索目录，DCC 会根据自身可执行文件的路径进行设置，如果路径查找失败，则会在环境变量 HIP_ROCCLR_HOME、HIP_PATH、HSA_PATH 等中继续查找。

developer.sourcefind.cn

Ldflags 主要用于链接参数，其中-L 表示系统库路径。若未指定优化等级，DCC 将默认使用-O3，并自动链接到一些常用库，例如-lgcc_s -lgcc -lpthread -lm -lrt 等。

3.3 DCC 调试选项

- 环境变量 HIPCC_VERBOSE=1

  打印 DCC 最终执行的编译命令字符串。

- 环境变量 HIPCC_VERBOSE=2

  打印 HIP 相关的环境变量或环境信息，如 HIP_PATH、HIP_PLATFORM、HIP_COMPILER 等。

- 环境变量 HIPCC_VERBOSE=4

  打印在执行 DCC 之前输入的所有编译参数。

- --version

  打印 HIP 与编译器版本信息。

- --short-version

  打印 HIP 与编译器的版本信息，不执行编译操作。

- --cxxflags

  打印 DCC 为编译命令新增的 C++编译参数，不执行编译。例如-isystem 用于指定系统头文件搜索目录。

- --ldflags

  打印 DCC 对编译命令添加的链接参数，不执行编译。例如--driver-mode、-L 等。

3.4 DCC 环境变量支持

- LD_LIBRARY_PATH

  用于设置链接库的搜索路径。

- HCC_AMDGPU_TARGET、AMDGPU_TARGETS

  用于指定编译的目标架构，架构用之间使用分号隔开，例如：AMDGPU_TARGETS= “gfx906;gfx926”。

- HIPCC_VERBOSE

  用于控制调试信息的输出。

开发者社区

- HIPCC_COMPILE_FLAGS_APPEND

  用于设置额外的编译命令。

- HIPCC_LINK_FLAGS_APPEND

  用于设置额外的链接命令。

- HIP_CLANG_HCC_COMPAT_MODE

  当设置为 1 时表示允许 __fp16 作为函数参数和返回类型

- HIP_COMPILE_CXX_AS_HIP

  当设置为 0 时，将以普通 C++程序的方式进行编译，而不是作为 HIP 程序。

- DCC_AR_TOOL

  当设置为 1 时，DCC 内部将使用 ar 作为压缩和解压工具；否则，使用 llvm-ar。

4 DCC 宏定义

4.1 功能检查宏

- __has_builtin：用于判断编译器是否支持指定的内建函数。其语法为__has_builtin(builtin-name)，其中 builtin-name 为内建函数的名称。返回值为 1 或 0，分别表示支持或不支持该内建函数。请注意，__has_builtin 不应用于检测内置宏的支持，应改用#ifdef 进行判断。
- __has_constexpr_builtin：用于检查编译器是否支持某个 constexpr 内建函数，并能在常量表达式中使用。若需要在编译时确定某个内建函数的可用性并希望在常量表达式中使用，应使用__has_constexpr_builtin。其使用方法与__has_builtin 类似。
- __has_feature 和__has_extension：
  1. __has_feature(feature)：用于检查编译器是否支持某个语言特性，例如右值引用（rvalue references）、可变参数模板（variadic templates）、static_assert 等。
  2. __has_extension(extension)：用于检查编译器是否支持某个扩展特性，例如__block、__attribute__((cleanup()))等。

  这两个宏都通过将待检查的特性或扩展作为参数传递，返回一个常量整数，

表示是否支持该特性或扩展。

- __has_cpp_attribute：用于检查编译器是否支持指定的 C++特性属性（attribute）。例如，可以使用此宏检查编译器是否支持 C++11 引入的[[noreturn]]属性。
- __has_c_attribute：用于检测编译器是否支持指定的 C 特性属性。例如，可用于检查 C11 引入的_Alignas 属性，该属性用于内存对齐。
- __has_attribute：用于检查编译器是否支持特定的属性（attribute）。
- __has_declspec_attribute：用于检测 Microsoft 扩展中的属性（attribute），如使用__declspec 关键字的属性。
- __is_identifier：用于检查给定的字符串是否为合法的标识符（identifier）。如果参数是有效的标识符，该宏返回 1；否则，返回 0。

4.2 头文件检查宏

- __has_include 和__has_include_next：用于判断某个头文件是否存在。

  __has_include：在标准头文件搜索路径中查找给定头文件

  __has_include_next：从当前位置开始在剩余头文件搜索路径中查找。

  这两个宏仅用于#if 或#elif 指令中，不能用于其它上下文中。同时，这两个宏不会实际包含头文件，仅用于判断头文件存在性。
- __has_warning：用于检查编译器是否支持产生警告信息。如果支持，该宏展开为数字 1，否则展开为数字 0。

4.3 内置宏

- DCC_VERSION_MAJOR：定义 DCC 主版本号。
- DCC_VERSION_MINOR：定义 DCC 次版本号。
- DCC_VERSION_PATCH：定义 DCC 补丁版本号。
- DCC_VERSION_GITHASH：定义 DCC Git 提交哈希值。
- INCLUDE_STD_FILESYSTEM_EXPERIMENTAL：用于指示是否需要包含实验性的标准文件系统库头文件。该宏的定义决定了 DCC 在编译时是否包含<experimental/filesystem>头文件。
- __BASE_FILE__：在当前文件中定义该宏时，其的值为当前文件完整路径名。

- __FILE_NAME__：在当前文件中定义该宏时，其值为当前文件的名称（不包括路径）。
- __COUNTER__：编译器内置的计数器变量，每次调用时其值自动递增 1。可用于在代码中生成唯一标识符。
- __INCLUDE_LEVEL__：在处理#include 文件或指令时递增的计数器。当编译器处理顶层源文件时，该计数器的值为 0。
- __TIMESTAMP__：在当前文件中定义该宏时，其值为当前文件的最后修改时间。
- __clang__：若用 Clang 编译，则该宏存在，其值为 1；否则，该宏不存在。
- __clang_major__：定义 Clang 主版本号的整数，例如 Clang 10.x.x 的主版本号为 10。
- __clang_minor__：定义 Clang 次版本号的整数，例如 Clang 10.2.x 的次版本号为 2。
- __clang_patchlevel__：定义 Clang 补丁版本号的整数，例如 Clang 10.2.1 的补丁版本号为 1。
- __clang_version__：字符串，表示当前 Clang 版本号。
- __clang_literal_encoding__：若当前编译模式为 C++17 或更高版本，则定义为一个整数，表示字面值编码类型。在 C++17 中，该编码类型包括 UTF-8、UTF-16 和 UTF-32；在 C++20 中，还包括 UTF-8 和 char8_t。
- __clang_wide_literal_encoding__：若当前编译模式为 C++17 或更高版本，则定义为一个整数，表示宽字面值(wide literal)编码类型。在 C++17 中，该编码类型包括 UTF-16 和 UTF-32；在 C++20 中，还包括 char16_t 和 char32_t。

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区